Introduce Code LLMs & Agents

刘芳 / Flora

Overview:

- AI Fundamentals: Transfromer Architecture(Encoder vs Decoder)
- The Ecosystem: AI Coding Agents vs Code LLMs
- Leading Coding LLMs(Large Language Modes)
- Prompt Engineering Principle & Techniques

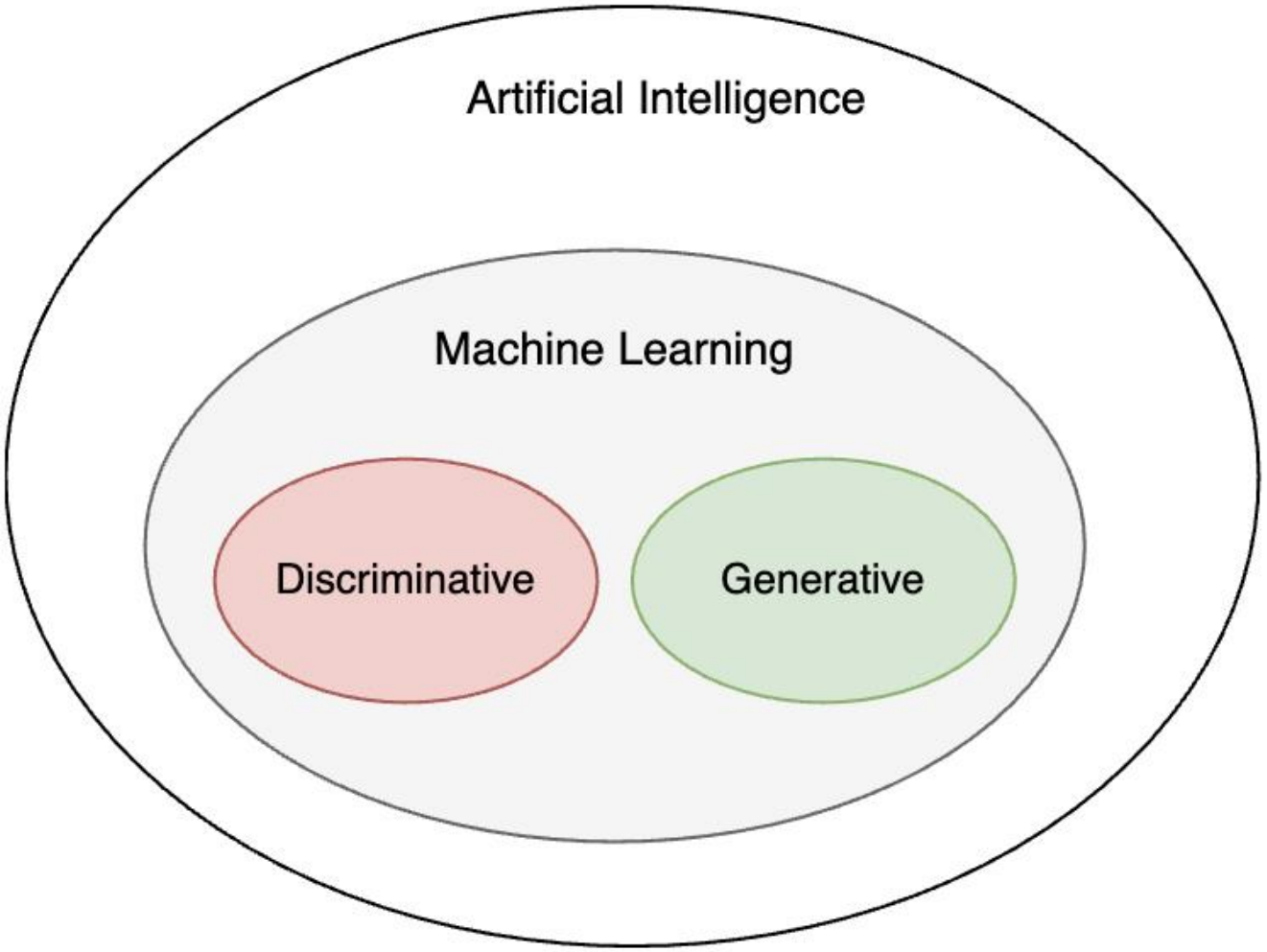


Figure 1: The relationship between AI and ML

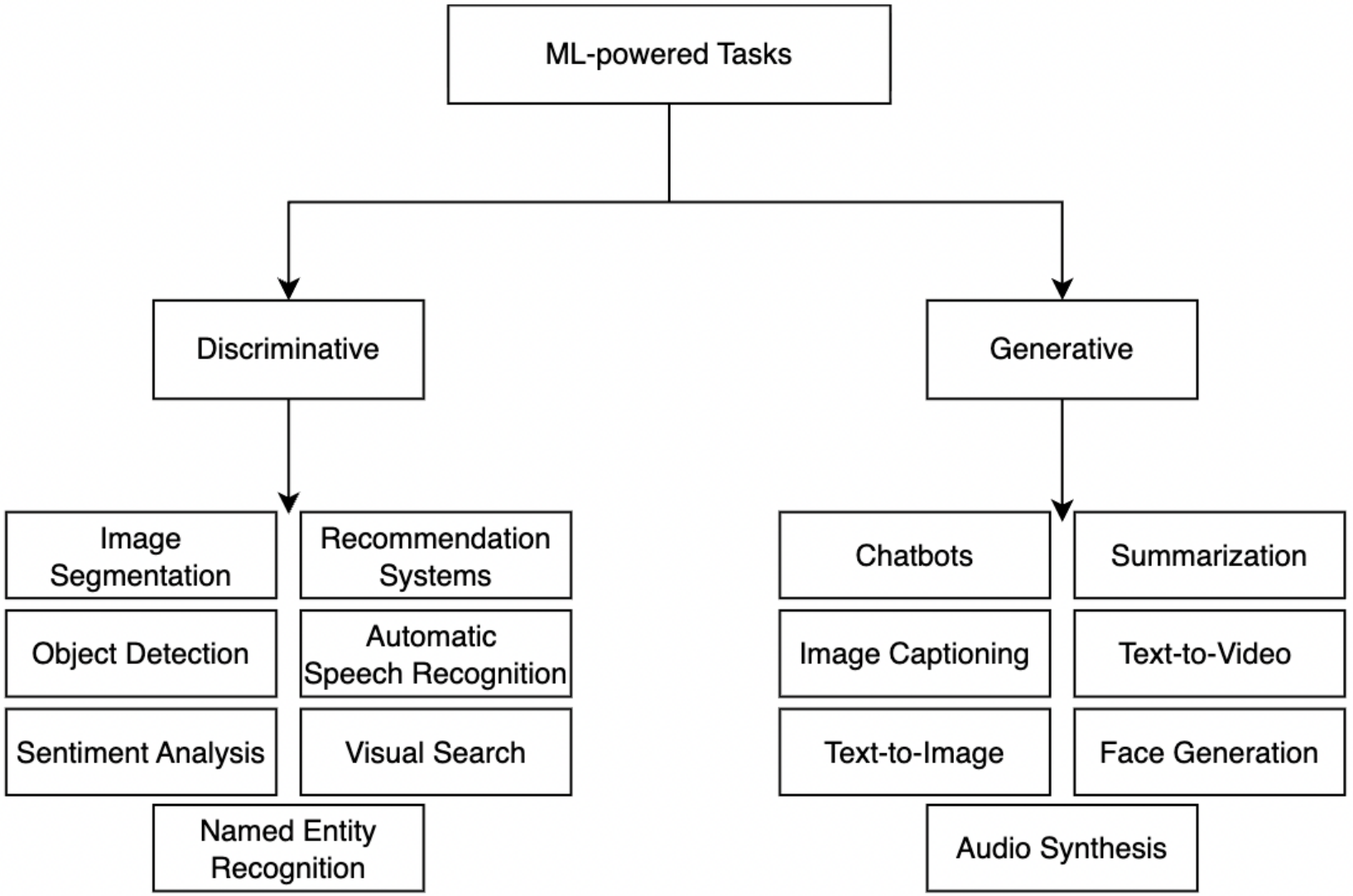


Figure 2: Popular ML-powered tasks

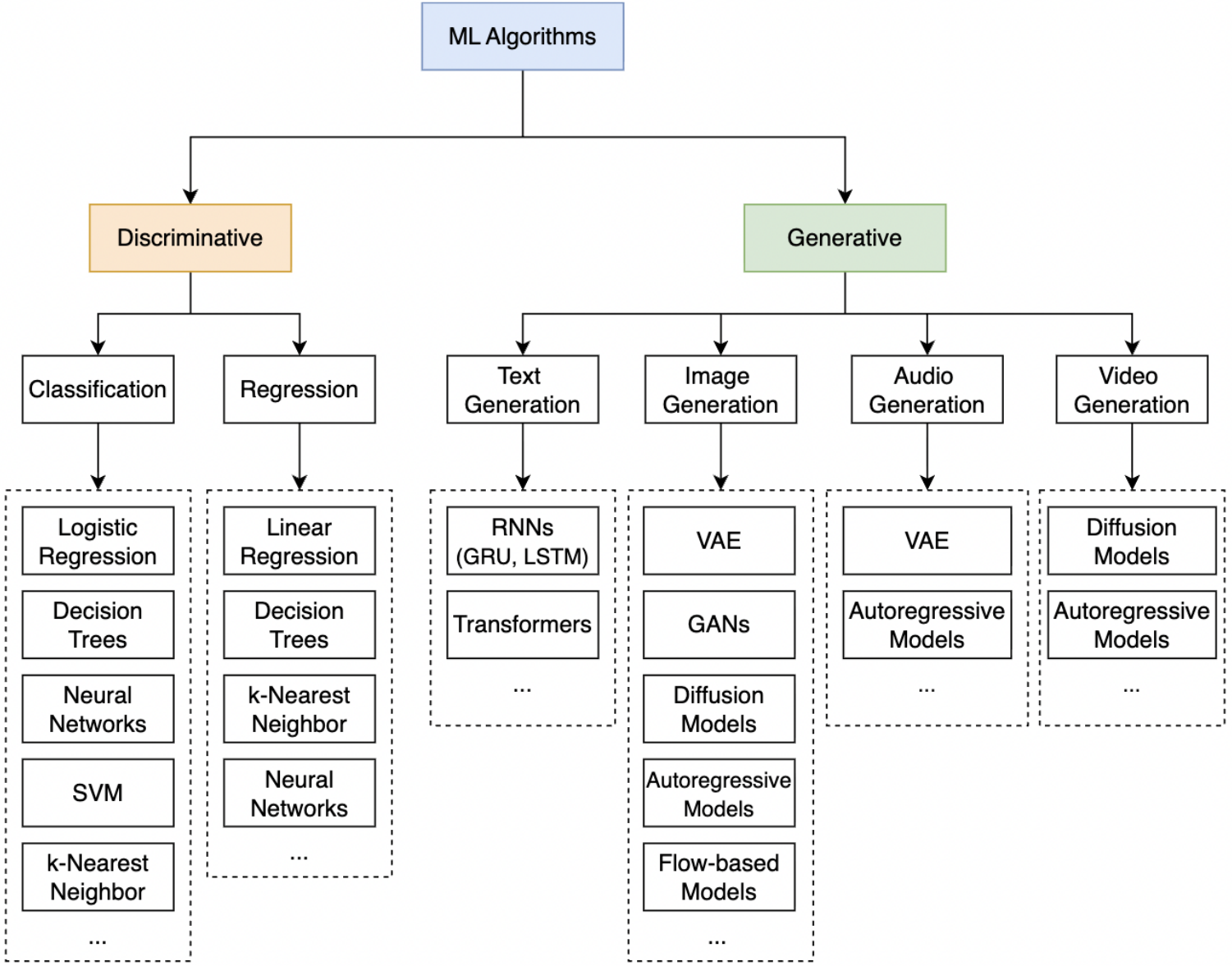
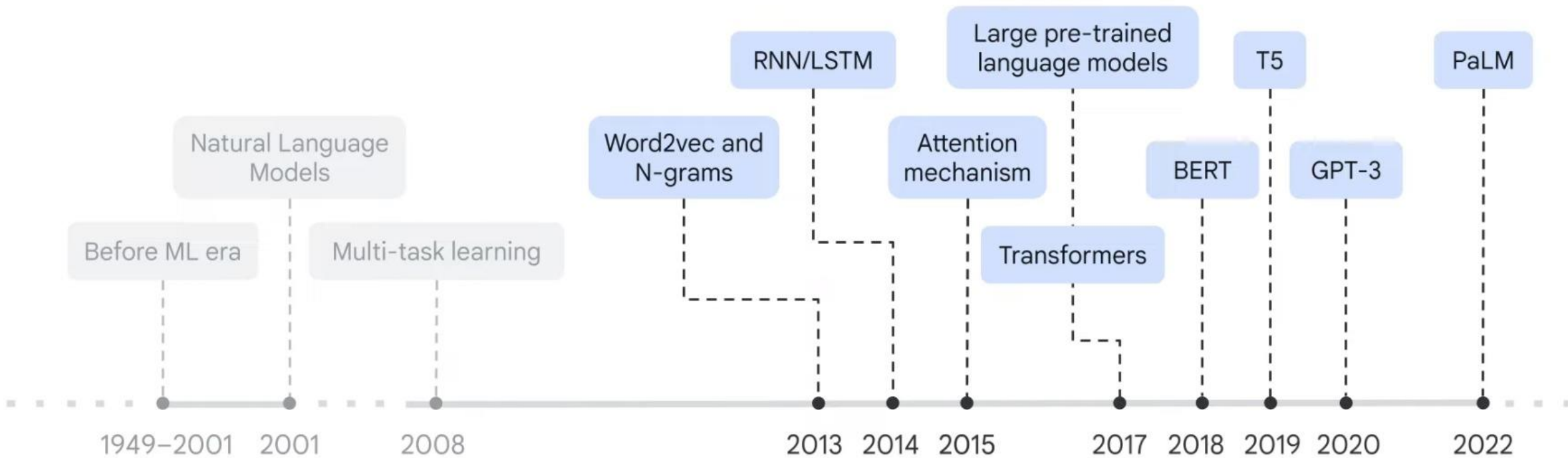


Figure 8: Common ML algorithms

Language modeling history

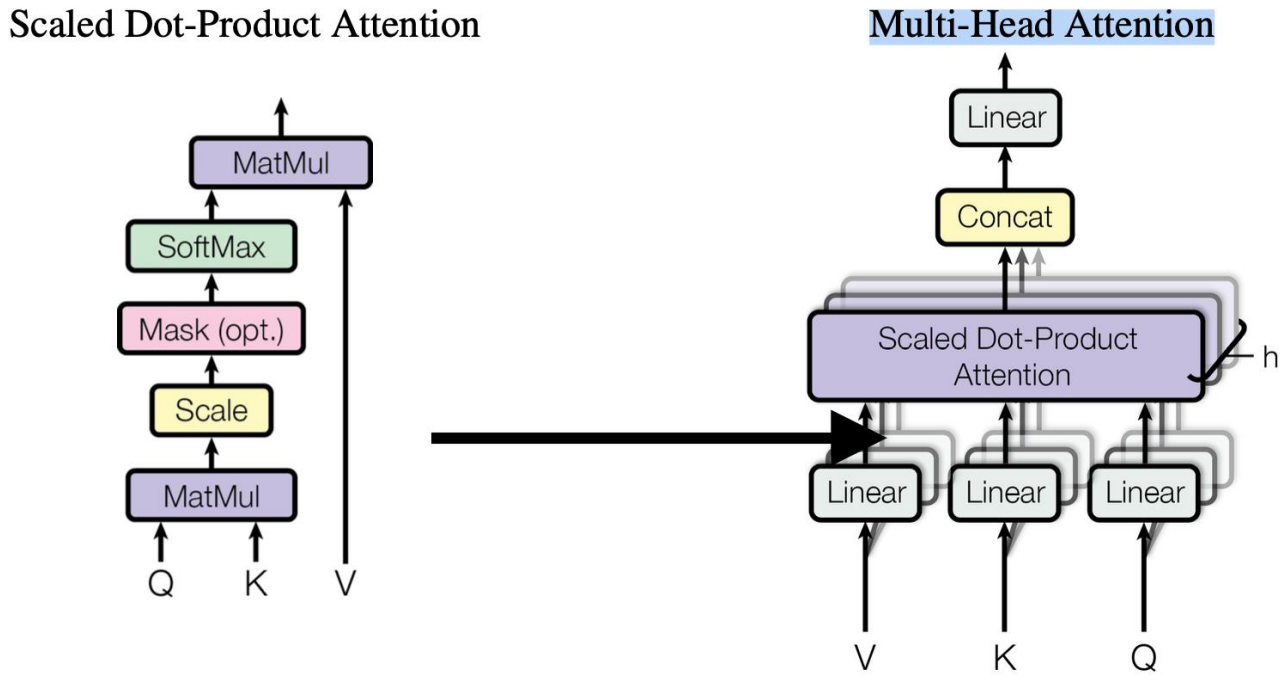


Code LLMs History

Era	Key Developments
Pre-2015: The **Statistical** Era	N-gram models, Hidden Markov Models (HMMs).
2015–2017: The **RNN & LSTM** Era	RNNs and LSTMs for code completion
2018–2019: The **Embeddings** Era	code2vec (2018): Represented code as paths in an AST. CuBERT (2019): Google's early BERT for code. CodeSearchNet (2019): A massive dataset that enabled future training.
2020: The Pre-**training** Era	CodeBERT (Feb 2020): Microsoft’s influential model. GPT-3 (May 2020): Showed emergent code generation without explicit code training.
2021: The **Generative** Breakthrough (Decoders)	Codex (July 2021): The engine behind GitHub Copilot. CodeT5 (Sept 2021): A powerful Encoder-Decoder model by Salesforce
2022: Competition & Scale	AlphaCode (Feb 2022): DeepMind’s model that solved competitive programming problems. InCoder (Facebook) & CodeGen (Salesforce): Popularized open-weights code generation. The Stack: BigCode released the first ethically sourced, massive code dataset.
2023: The Open Source Explosion	StarCoder (May), Code Llama (Aug), DeepSeek Coder V1 (Nov), GPT-4
2024 -- now: Practical	Claude 3.5 Sonnet (June 2024),DeepSeek Coder V2 (2024), Qwen3-Coder, Claude Sonnet 4.5(2025.9), Google 3 Pro , GLM-4.7, Kimi K2(2507)

Transformer: Attention is all you need(2017)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

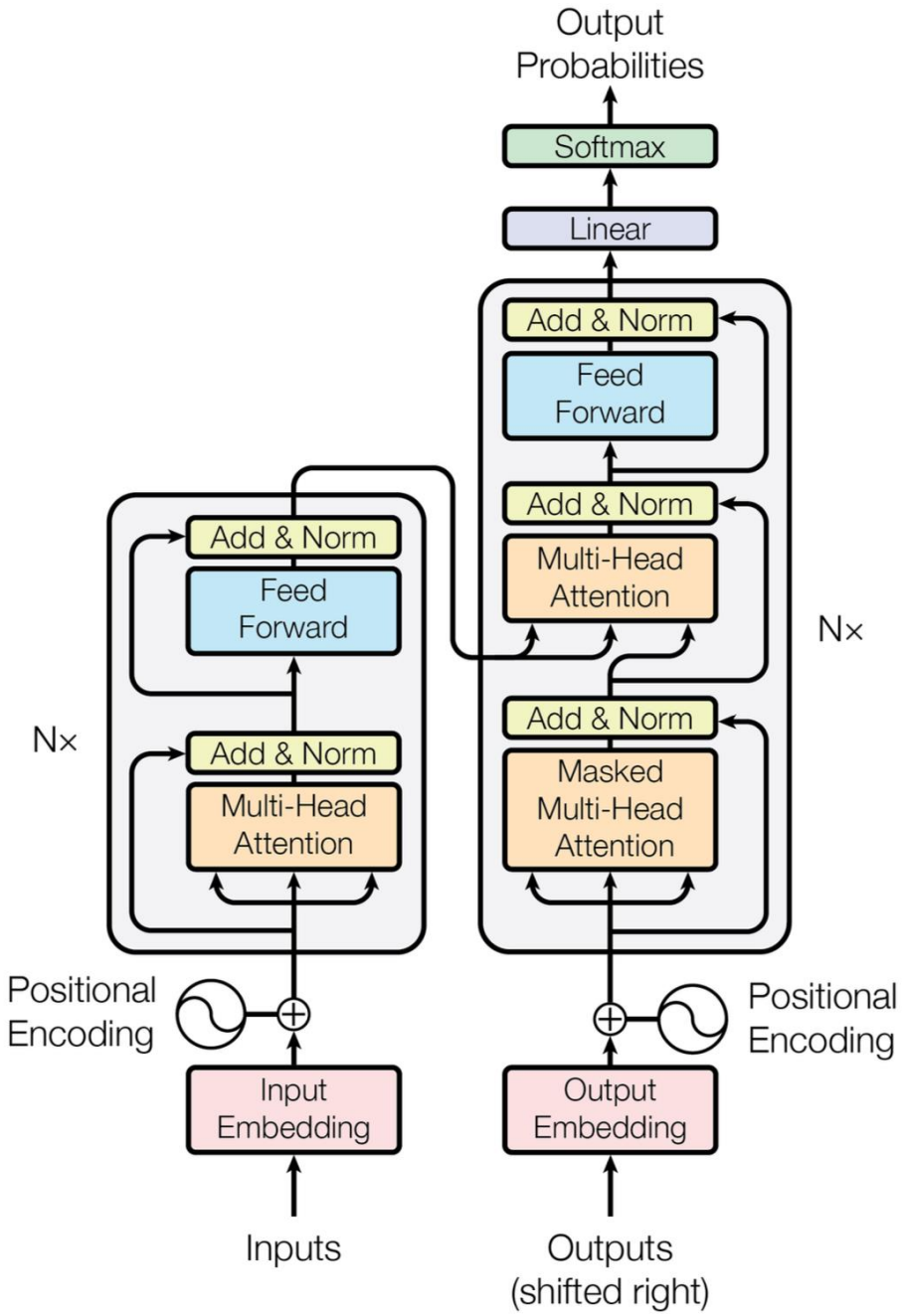


Figure 1: The Transformer - model architecture.

Encoder VS Decoder

Encoder: Reads and understands the input text, creating a rich representation of its meaning

use cases:

1. Classification tasks(sentiment analysis, spam detection)
2. Creates contextual embeddings that capture the full meaning
3. Good at understanding relationships between all parts of the input

Decoder: Generates new text, one token at a time, based on previous tokens

use cases:

1. Text Generation
2. Creative writing
3. Chatbots and conversational AI
4. Code generation

Encoder-only (BERT):

Input → [Encoder] → Understanding/Classification

Decoder-only (GPT):

Input/Previous tokens → [Decoder] → Generate next token

Encoder-Decoder (T5):

Input → [Encoder] → Context → [Decoder] → Output

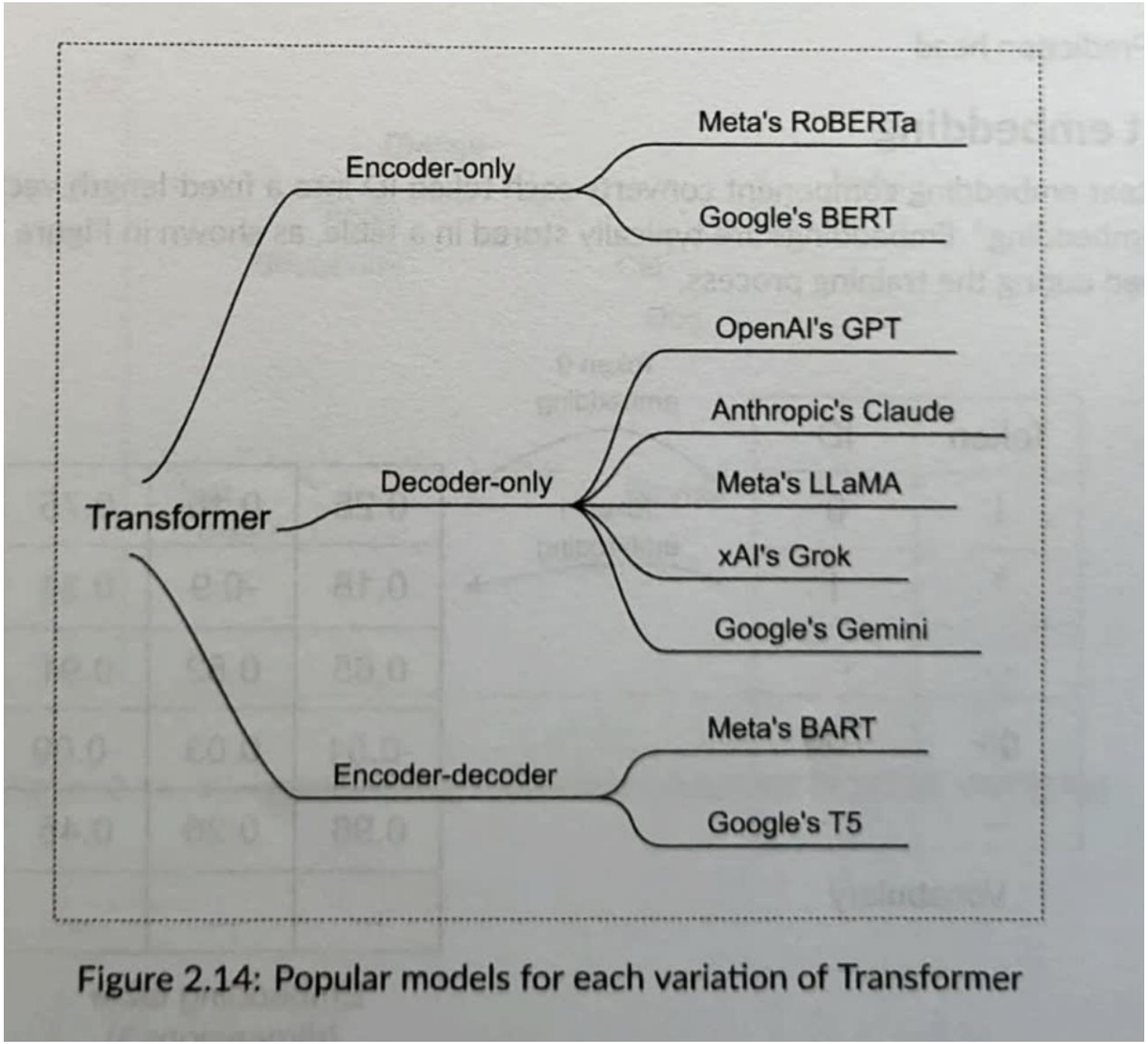


Figure 2.14: Popular models for each variation of Transformer

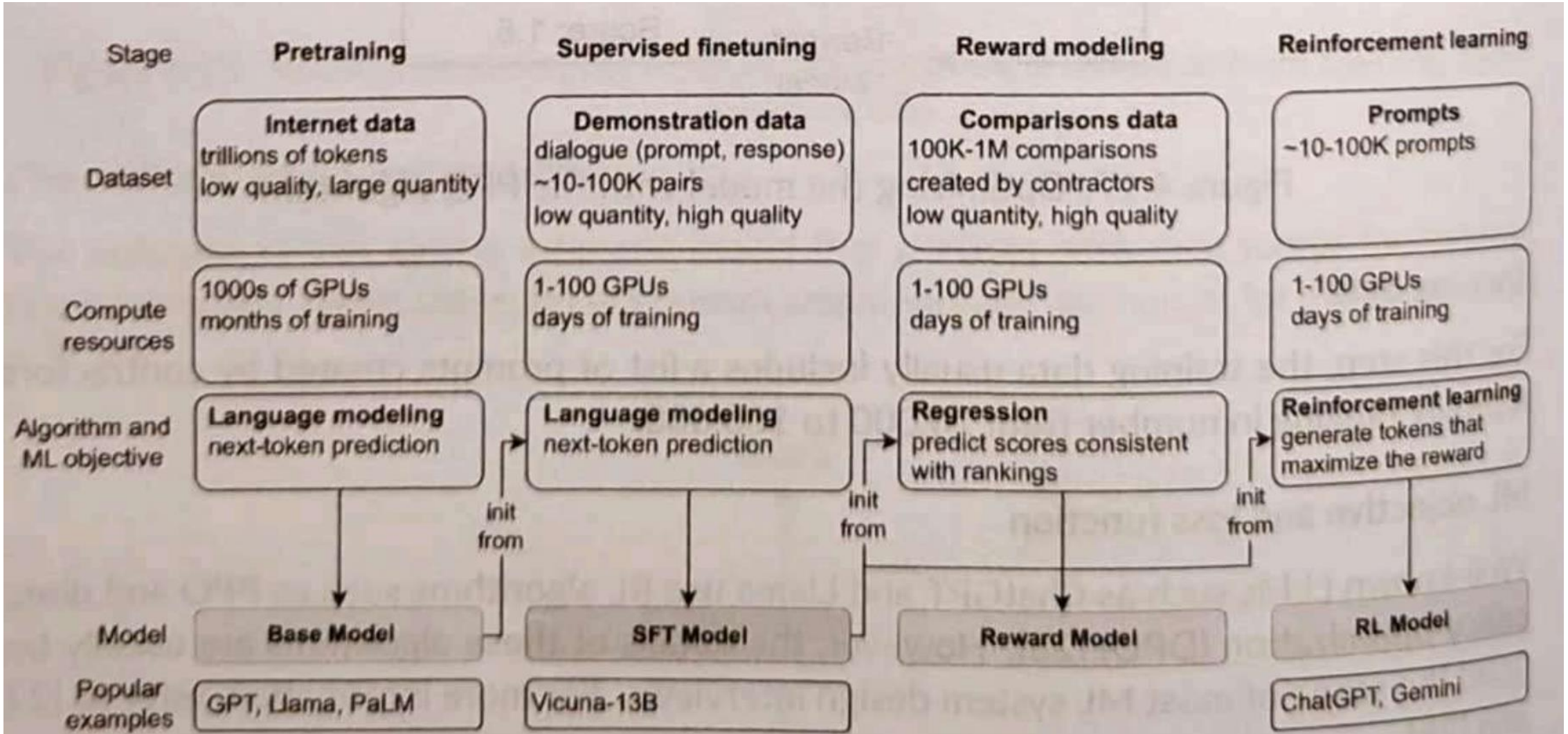


Figure 4.18: Summary of LLM training, inspired by [29]

https://playground.tensorflow.org/#activation=tanh&regularization=L2&batchSize=10&dataset=circle&regDataset=reg-plane&learningRate=0.03&regularizationRate=0&noise=0&networkShape=4,2&seed=0.37010&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false

AI Coding Tools & Agents

International

1. **Claude**(Anthropic, Opus4.5(Planning) Sonnet4.5(Execution)

2. Cursor(Composer1, Grok Code, Kimi K2)

3. Antigravity(**Google's Gemini 3 models**
Gemini 3 Pro
Gemini 3 Deep Think
Gemini 3 Flash
Anthropic's Claude 4.5 models:
OpenAI variant:GPT-OSS-120B)

4. CodeX(OpenAI, **GPT-5.2-Codex GPT-5.1-Codex, GPT-5.1-Codex-Max, GPT-5.1-Codex-Max**)

5. Windsurf(GPT 5.2, claude opus, Sonnet, SWE-1.5)

6. Kilo Code(Amazon)

Chinese

1. **TRAE**(Gemini-3-pro-preview, Gemini-3-pro-preview(200k), Gemini-2.5-Pro, Kimi-K2-0905, GPT-5.2/GPT5.1, Grok-4, DeepSeek-V3.1 , Gemini-2.5-Flash)

2**. Z.ai**(Claude opus 4.5, GLM-4.7, GPT 5.2)

3. **Kimi Code**(kimi K2)

4. Qoder (Alibaba)

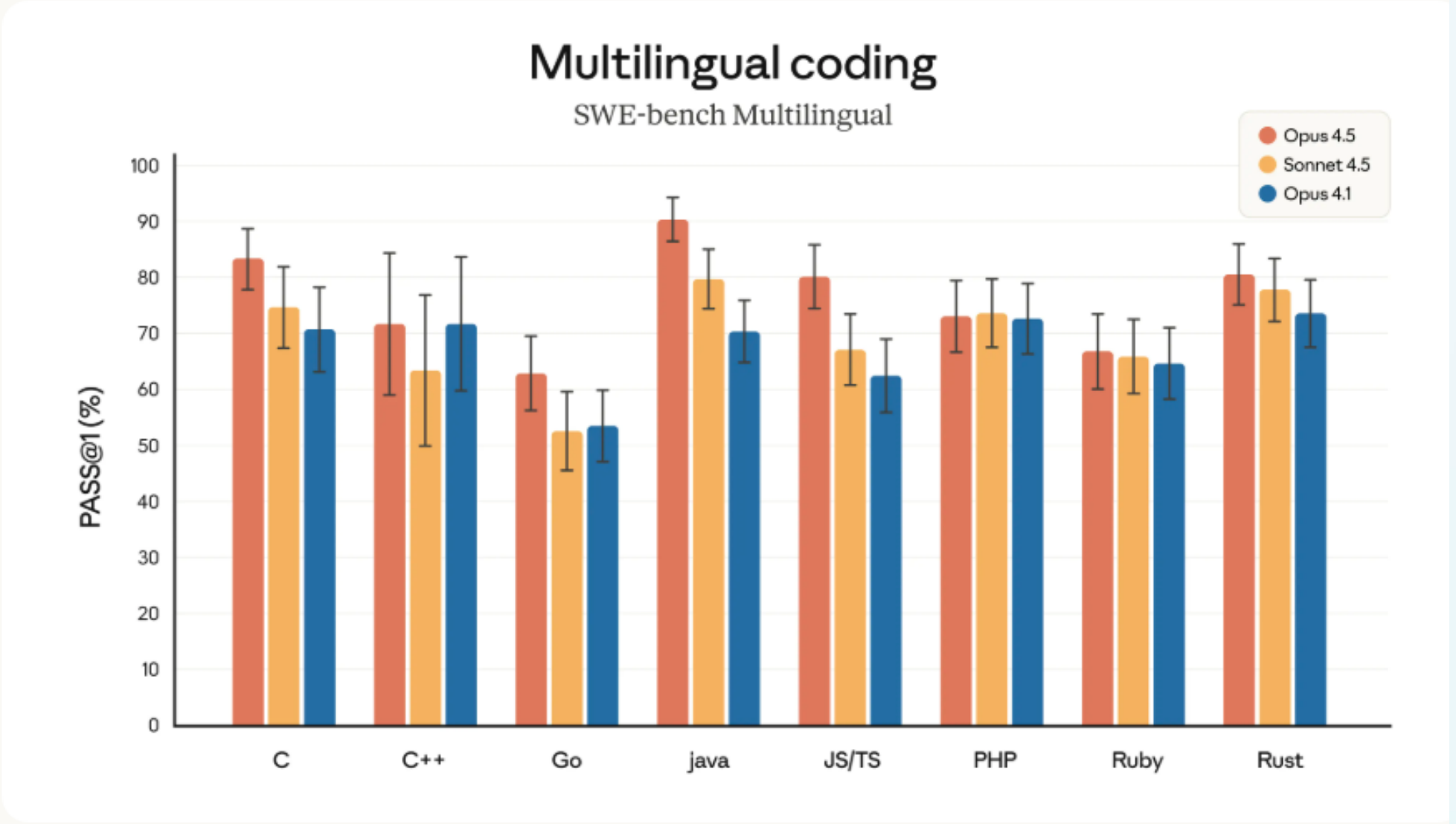
Code Large Language Models

- Opus 4.5(Planning) & Sonnet 4.5(Execution)
- Gemini 3
- GPT-5.2-Codex
- Kimi K2
- GLM-4.7
- Composer 1(Cursor)
- QWEN3-Coder

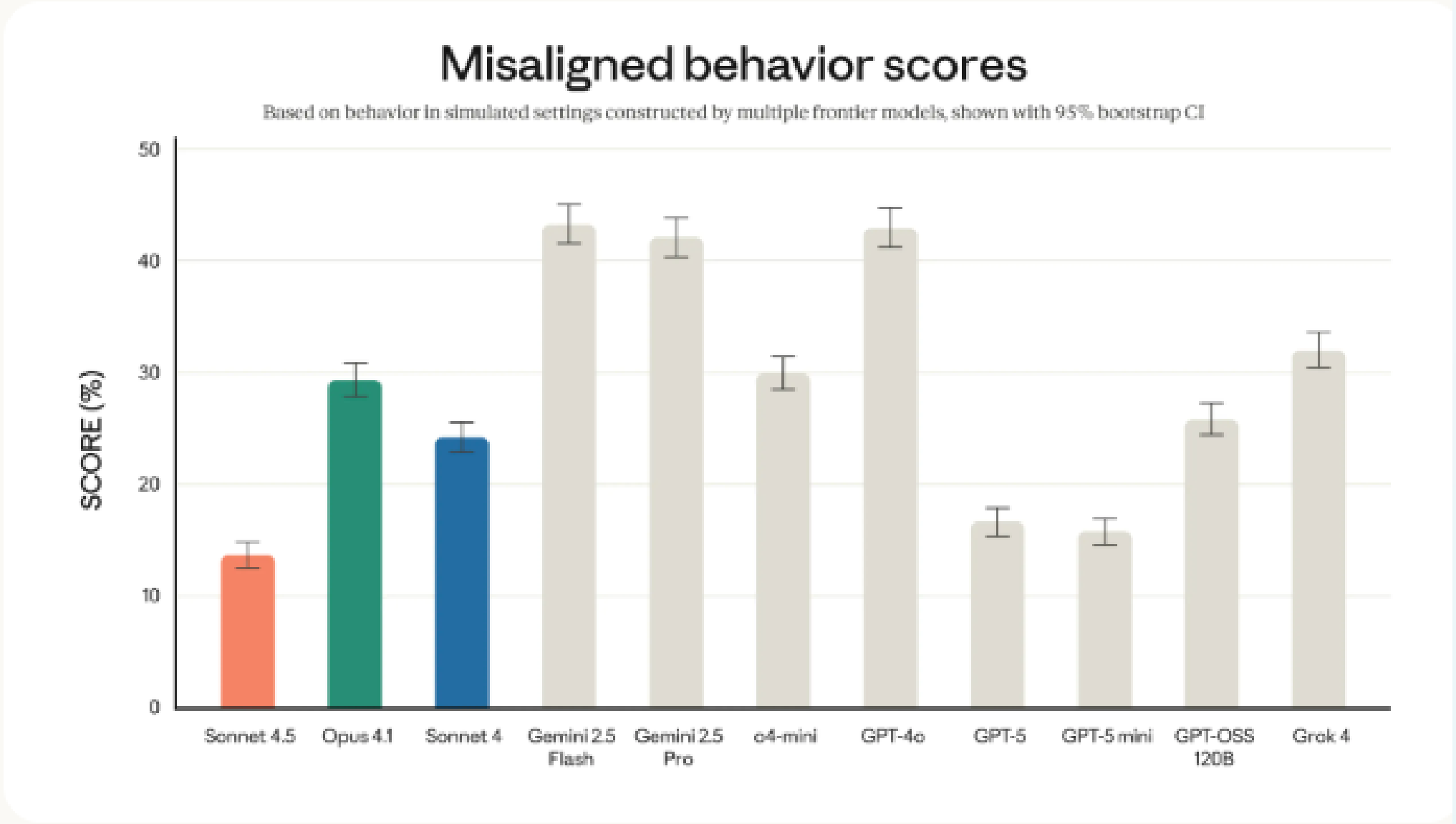
Anthropic Claude Sonnet 4.5(2025.09)

	Claude Sonnet 4.5	Claude Opus 4.1	Claude Sonnet 4	GPT-5	Gemini 2.5 Pro
Agentic coding *SWE-bench Verified*	**77.2%** **82.0%** with parallel test-time compute	74.5% 79.4% with parallel test-time compute	72.7% 80.2% with parallel test-time compute	72.8% GPT-5 74.5% GPT-5-Codex	67.2%
Agentic terminal coding *Terminal-Bench*	**50.0%**	46.5%	36.4%	43.8%	25.3%
Agentic tool use *τ2-bench*	Retail **86.2%** Airline **70.0%** Telecom **98.0%**	Retail **86.8%** Airline 63.0% Telecom 71.5%	Retail 83.8% Airline 63.0% Telecom 49.6%	Retail 81.1% Airline 62.6% Telecom 96.7%	— — —
Computer use *OSWorld*	**61.4%**	44.4%	42.2%	—	—
High school math competition *AIME 2025*	100% (python) 87.0% (no tools)	78.0%	70.5%	99.6% (python) 94.6% (no tools)	88.0%
Graduate-level reasoning *GPQA Diamond*	83.4%	81.0%	76.1%	85.7%	**86.4%**
Multilingual Q&A *MMMLU*	89.1%	**89.5%**	86.5%	89.4%	—
Visual reasoning *MMMU (validation)*	77.8%	77.1%	74.4%	**84.2%**	82.0%
Financial analysis *Finance Agent*	**55.3%**	50.9%	44.5%	46.9%	29.4%

SWE-bench Multilingual | Aider Polyglot | BrowseComp-Plus | Vending-Bench



Opus 4.5 writes better code, leading across 7 out of 8 programming languages on SWE-bench Multilingual.



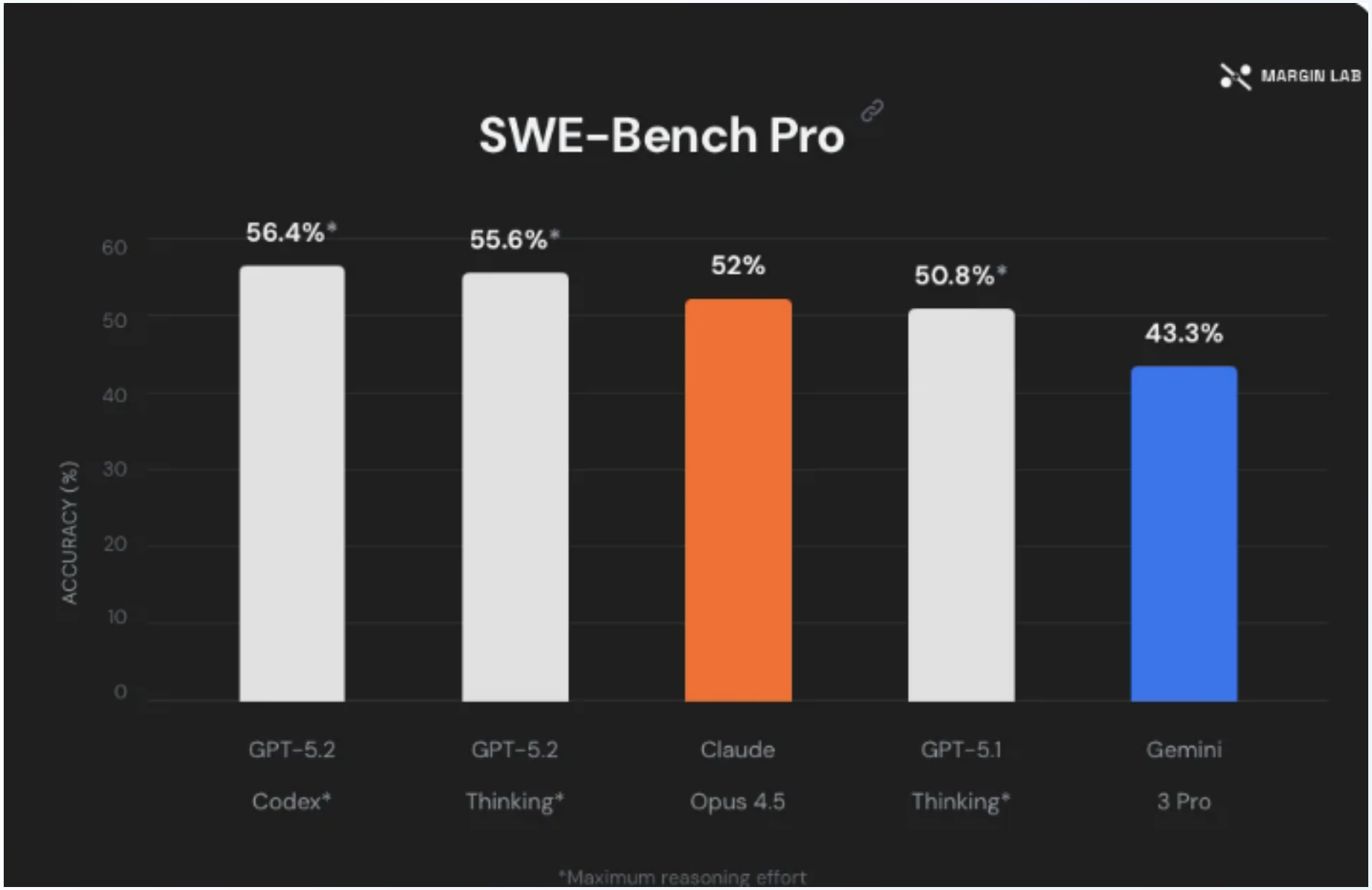
Overall misaligned behavior scores from an automated behavioral auditor (lower is better). Misaligned behaviors include (but are not limited to) deception, sycophancy, power-seeking, encouragement of delusions, and compliance with harmful system prompts. More details can be found in the Claude Sonnet 4.5 system card.

Google 3 Pro 2025.11.18

Benchmark	Description		Gemini 3 Pro	Gemini 2.5 Pro	Claude Sonnet 4.5	GPT-5.1
Humanity's Last Exam	Academic reasoning	No tools	**37.5%**	21.6%	13.7%	26.5%
		With search and code execution	**45.8%**	—	—	—
ARC-AGI-2	Visual reasoning puzzles	ARC Prize Verified	**31.1%**	4.9%	13.6%	17.6%
GPQA Diamond	Scientific knowledge	No tools	**91.9%**	86.4%	83.4%	88.1%
AIME 2025	Mathematics	No tools	**95.0%**	88.0%	87.0%	94.0%
		With code execution	**100%**	—	**100%**	—
MathArena Apex	Challenging Math Contest problems		**23.4%**	0.5%	1.6%	1.0%
MMMU-Pro	Multimodal understanding and reasoning		**81.0%**	68.0%	68.0%	76.0%
ScreenSpot-Pro	Screen understanding		**72.7%**	11.4%	36.2%	3.5%
CharXiv Reasoning	Information synthesis from complex charts		**81.4%**	69.6%	68.5%	69.5%
OmniDocBench 1.5	OCR	Overall Edit Distance, lower is better	**0.115**	0.145	0.145	0.147
Video-MMMU	Knowledge acquisition from videos		**87.6%**	83.6%	77.8%	80.4%
LiveCodeBench Pro	Competitive coding problems from Codeforces, ICPC, and IOI	Elo Rating, higher is better	**2,439**	1,775	1,418	2,243
Terminal-Bench 2.0	Agentic terminal coding	Terminus-2 agent	**54.2%**	32.6%	42.8%	47.6%
SWE-Bench Verified	Agentic coding	Single attempt	76.2%	59.6%	**77.2%**	76.3%
τ2-bench	Agentic tool use		**85.4%**	54.9%	84.7%	80.2%
Vending-Bench 2	Long-horizon agentic tasks	Net worth (mean), higher is better	**$5,478.16**	$573.64	$3,838.74	$1,473.43
FACTS Benchmark Suite	Held out internal grounding, parametric, MM, and search retrieval benchmarks		**70.5%**	63.4%	50.4%	50.8%
SimpleQA Verified	Parametric knowledge		**72.1%**	54.5%	29.3%	34.9%
MMMLU	Multilingual Q&A		**91.8%**	89.5%	89.1%	91.0%
Global PIQA	Commonsense reasoning across 100 Languages and Cultures		**93.4%**	91.5%	90.1%	90.9%
MRCR v2 (8-needle)	Long context performance	128k (average)	**77.0%**	58.0%	47.1%	61.6%
		1M (pointwise)	**26.3%**	16.4%	not supported	not supported

For details on our evaluation methodology please see deepmind.google/models/evals-methodology/gemini-3-pro

GPT-5.2-Codex(2025.12.18)



Kimi K2(25.07)

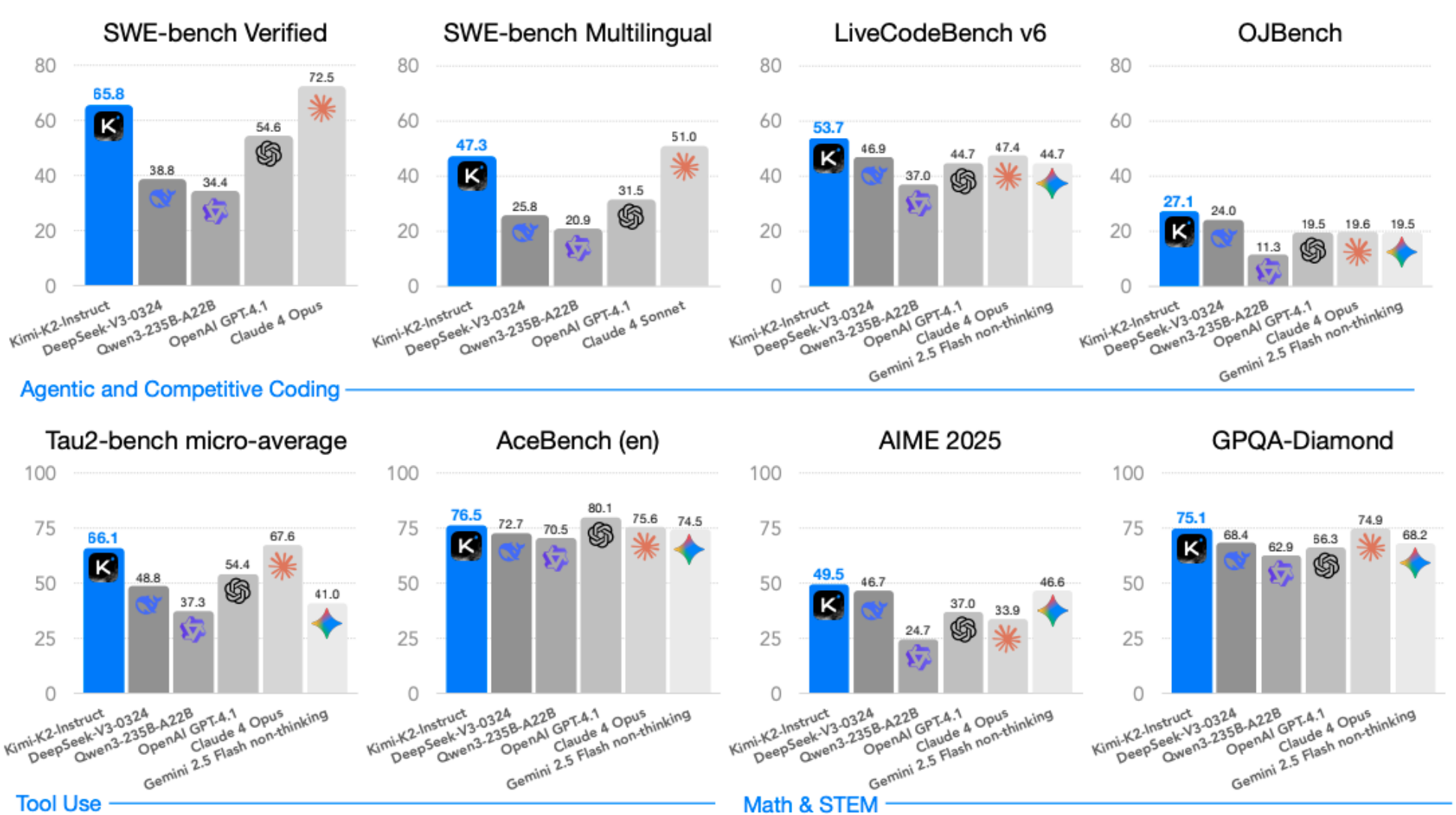


Figure 1: Kimi K2 main results.[2]

Coding Tasks

SWE-bench Verified
SWE-bench Multilingual
LiveCodeBench v6 (Pass@1
OJBench (Online Judge Benchmark (Pass@1))
HumanEval(basic functions or interview-style questions)

Tool Use Tasks:

AceBench(Agentic Context Engineering)

Math & STEM:

AIME(**American Invitationa Mathematics Examination**)
GPQA-Diamond(**Graduate-Level Google-Proof Q&A**)

Kimi K2(25.07)

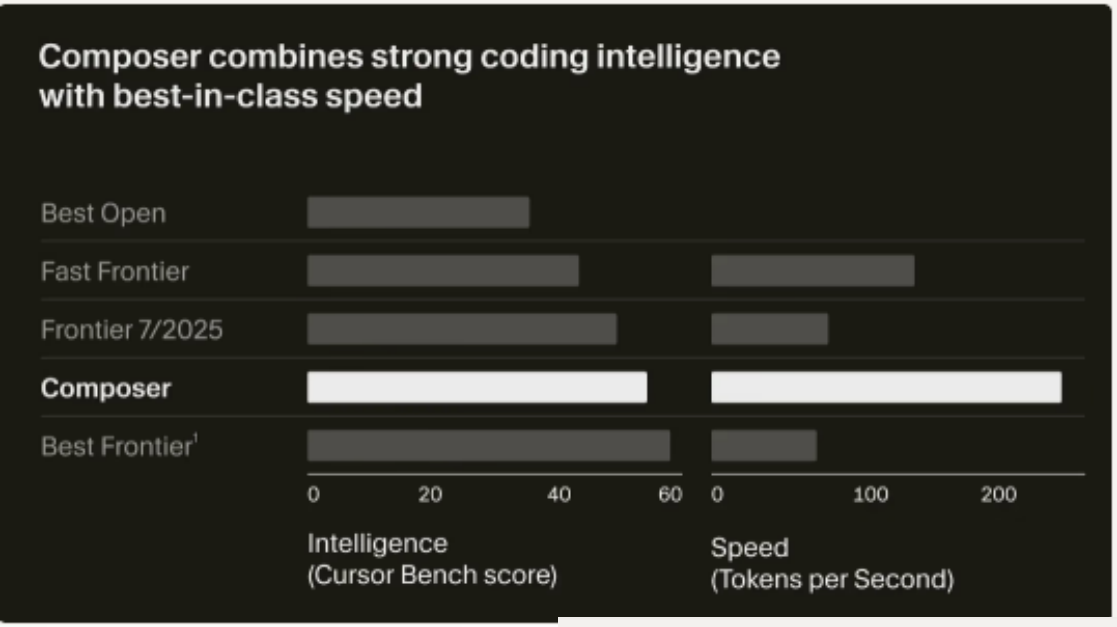
Benchmark	Open Source			Proprietary			
	Kimi-K2-Instruct	DeepSeek-V3-0324	Qwen3-235B-A22B	Claude Sonnet 4	Claude Opus 4	GPT-4.1	Gemini 2.5 Flash
Coding Tasks							
LiveCodeBench v6 (Pass@1)	**53.7**	46.9	37.0	48.5	47.4	44.7	44.7
OJBench (Pass@1)	**27.1**	24.0	11.3	15.3	19.6	19.5	19.5
MultiPL-E (Pass@1)	**85.7**	83.1	78.2	88.6	**89.6**	86.7	85.6
SWE-bench Verified *Agentless-Single-Patch* (Pass@1)	**51.8**	36.6	39.4	50.2	**53.0**	40.8	32.6
SWE-bench Verified *Agentic-Single-Attempt* (Pass@1)	**65.8**	38.8	34.4	**72.7***	72.5*	54.6	—
SWE-bench Verified *Agentic-Multi-Attempt* (Pass@1)	**71.6**	—	—	**80.2***	79.4*	—	—
SWE-bench Multilingual (Pass@1)	**47.3**	25.8	20.9	**51.0**	—	31.5	—
Multi-SWE-bench (Pass@1)	**18.3**	8.0	9.0	**29.2**	—	11.7	14.0
SWE-Lancer (Pass@1)	**39.1**	30.5	24.1	**40.8**	—	23.0	38.5
Paper Bench *Code-Dev* (Acc.)	**27.8**	12.2	13.2	**43.3**	—	29.9	5.7
Terminal Bench *In-House* (Acc.)	**30.0**	—	—	35.5	**43.2**	8.3	—
Terminal Bench *Terminus* (Acc.)	**25.0**	16.3	6.6	—	—	**30.3**	16.8
Aider-Polyglot (Acc.)	60.0	55.1	**61.8**	56.4	**70.7**	52.4	44.0
Tool Use Tasks							
Tau2 retail (Avg@4)	**70.6**	69.1	57.0	75.0	**81.8**	74.8	64.3
Tau2 airline (Avg@4)	**56.5**	39.0	26.5	55.5	**60.0**	54.5	42.5
Tau2 telecom (Avg@4)	**65.8**	32.5	22.1	45.2	57.0	38.6	16.9
AceBench (Acc.)	**76.5**	72.7	70.5	76.2	75.6	**80.1**	74.5
Math & STEM Tasks							
AIME 2024 (Avg@64)	**69.6**	59.4*	40.1*	43.4	48.2	46.5	61.3
AIME 2025 (Avg@64)	**49.5**	46.7	24.7*	33.1*	33.9*	37.0	46.6
MATH-500 (Acc.)	**97.4**	94.0*	91.2*	94.0	94.4	92.4	95.4
HMMT 2025 (Avg@32)	**38.8**	27.5	11.9	15.9	15.9	19.4	34.7
CNMO 2024 (Avg@16)	74.3	**74.7**	48.6	60.4	57.6	56.6	**75.0**
PolyMath-en (Avg@4)	**65.1**	59.5	51.9	52.8	49.8	54.0	49.9
ZebraLogic (Acc.)	**89.0**	84.0	37.7*	79.7	59.3	58.5	57.9
AutoLogi (Acc.)	**89.5**	88.9	83.3*	89.8	86.1	88.2	84.1
GPQA-Diamond (Avg@8)	**75.1**	68.4*	62.9*	70.0*	74.9*	66.3	68.2
SuperGPQA (Acc.)	**57.2**	53.7	50.2	55.7	56.5	50.8	49.6
Humanity's Last Exam (Acc.)	4.7	5.2	**5.7**	5.8	**7.1**	3.7	5.6

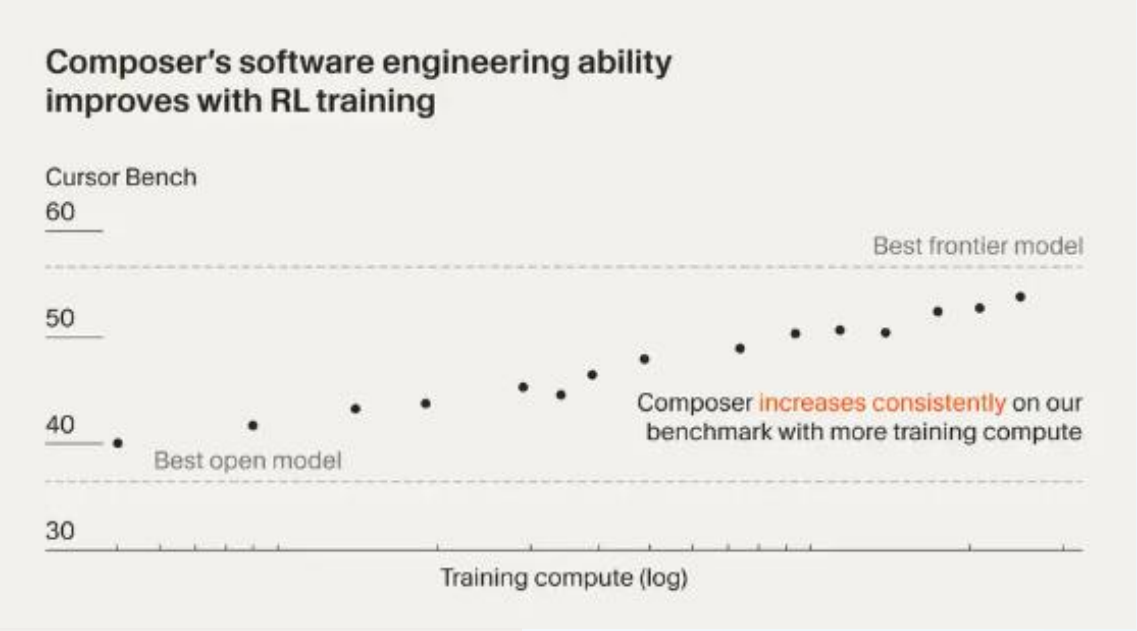
GLM-4.7(2025-12-22)

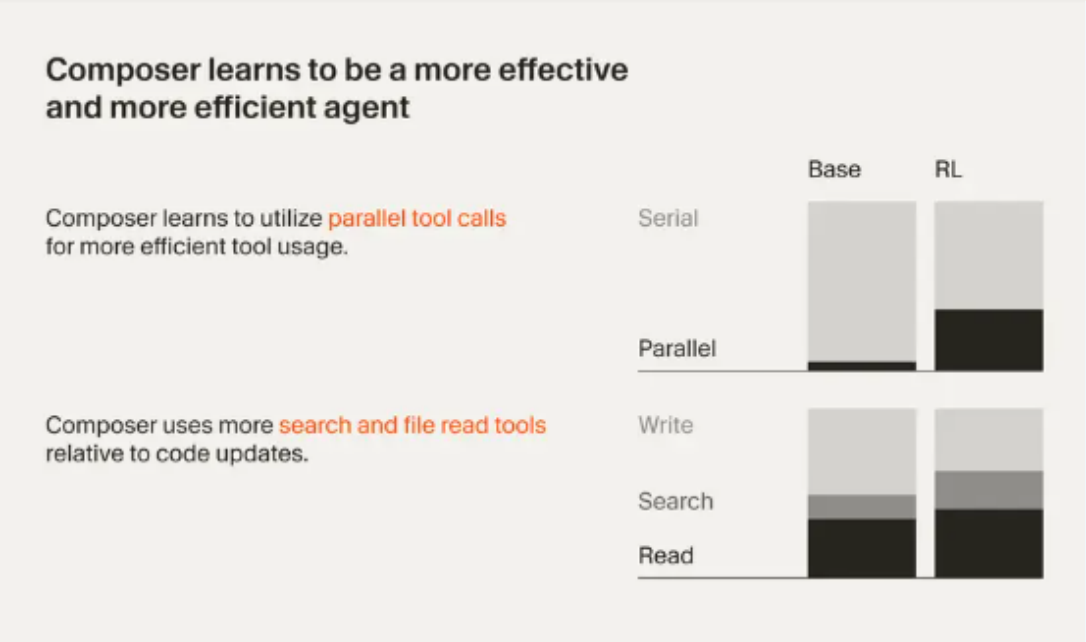
Benchmark	GLM-4.7	GLM-4.6	Kimi K2 Thinking	DeepSeek-V3.2	Gemini 3.0 Pro	Claude Sonnet 4.5	GPT-5 High	GPT-5.1 High
REASONING								
MMLU-Pro	84.3	83.2	84.6	85.0	**90.1**	88.2	87.5	87.0
GPQA-Diamond	85.7	81.0	84.5	82.4	**91.9**	83.4	85.7	88.1
HLE	24.8	17.2	23.9	25.1	**37.5**	13.7	26.3	25.7
HLE (w/ Tools)	42.8	30.4	44.9	40.8	**45.8**	32.0	35.2	42.7
AIME 2025	**95.7**	93.9	94.5	93.1	95.0	87.0	94.6	94.0
HMMT Feb. 2025	97.1	89.2	89.4	92.5	**97.5**	79.2	88.3	96.3
HMMT Nov. 2025	**93.5**	87.7	89.2	90.2	93.3	81.7	89.2	-
IMOAnswerBench	82.0	73.5	78.6	78.3	**83.3**	65.8	76.0	-
LiveCodeBench-v6	84.9	82.8	83.1	83.3	**90.7**	64.0	87.0	87.0
CODE AGENT								
SWE-bench Verified	73.8	68.0	71.3	73.1	76.2	**77.2**	74.9	76.3
SWE-bench Multilingual	66.7	53.8	61.1	**70.2**	-	68.0	55.3	-
Terminal Bench Hard	33.3	23.6	30.6	35.4	39.0	33.3	30.5	**43.0**
Terminal Bench 2.0	41.0	24.5	35.7	46.4	**54.2**	42.8	35.2	47.6
GENERAL AGENT								
BrowseComp	52.0	45.1	-	51.4	-	24.1	**54.9**	50.8
BrowseComp (w/ Context Manage)	67.5	57.5	60.2	**67.6**	59.2	-	-	-
BrowseComp-ZH	**66.6**	49.5	62.3	65.0	-	42.4	63.0	-
τ^2-Bench	87.4	75.2	74.3	85.3	**90.7**	87.2	82.4	82.7

Composer 1(Cursor, 2025.10.29)

4X faster than others， Intelligence. VS Speed







Qwen3-Coder(2025.7)

huybery@U-V76490MY-0133:~/coding-agent

100% 23% 16 GB 3.1 kB↓ 2.0 kB↑

❯ QWEN3-CODER

Benchmarks	Open Models			Proprietary Models	
	Qwen3-Coder 480B-A35B-Instruct	Kimi-K2 Instruct	DeepSeek-V3 0324	Claude Sonnet-4	OpenAI GPT-4.1
Agentic Coding					
Terminal-Bench	37.5	30.0	2.5	35.5	25.3
SWE-bench Verified					
w/ OpenHands, 500 turns	69.6	-	-	70.4	-
w/ OpenHands, 100 turns	67.0	65.4	38.8	68.0	48.6
w/ Private Scaffolding	-	65.8	-	72.7	63.8
SWE-bench Live	26.3	22.3	13.0	27.7	-
SWE-bench Multilingual	54.7	47.3	13.0	53.3	31.5
Multi-SWE-bench mini	25.8	19.8	7.5	24.8	-
Multi-SWE-bench flash	27.0	20.7	-	25.0	-
Aider-Polyglot	61.8	60.0	56.9	56.4	52.4
Spider2	31.1	25.2	12.8	31.1	16.5
Agentic Browser Use					
WebArena	49.9	47.4	40.0	51.1	44.3
Mind2Web	55.8	42.7	36.0	47.4	49.6
Agentic Tool Use					
BFCL-v3	68.7	65.2	64.7	73.3	62.9
TAU-Bench Retail	77.5	70.7	59.1	80.5	-
TAU-Bench Airline	60.0	53.5	40.0	60.0	-

Prompt Design Principle:

1. **Start Simple** : Begin with straightforward prompts and gradually introudce more complexity, Tools such as Cohere's playground allow you to easily test and adjust prompts as needed
2. **Break Down Complex Tasks (Divide and Conquer 分而治之)**
3. **Use Clear Instructions**: Clear, **Action-oriented** commands(Write, Summarize, Translate), use "###" to separated by delimiter
4. **Be Specific**: Format, style, outcomes, input, ouputs
5. Experiments with **Prompt Length**

Prompt Engineering Techniques:

1. Reasoning First
   - Before writing code, explain your approach
2. Chain-Of-Thought Prompting
   - Solve this step by step
   - Let's think step by step(Kojima et al. (2022))
   - You need first to write a step-by-step outline and then write the code
   - Break this into sub-problems, solve each, then combine(分而治之)
3. Few-Shot Prompting
   - Here are 2 examples: Example 1: [problem] → [solution], Example 2: [problem] → [solution], Now solve: [your task]
4. Role-Specific Prompting
   - You're a helpful code assistant, Write high-quality code to solve,please in-depth to help me to analyze the issue
   - You are an expert software engineer with 10+ years of experience, please In-depth to help me to analyze the issue
5. User-Context Prompting
6. Professional Term use English
7. Reference is Important(追本溯源，第一性原理)

8. Explicit Constraints
"Requirements: O(n) time, handle empty input, include type hints"

9. Test-Driven
"Your solution must pass these tests: [test cases]"

10. Self-Debugging
"After writing, review your code for bugs and provide improved version."

11. Decomposition
"Break this into sub-problems, solve each, then combine."

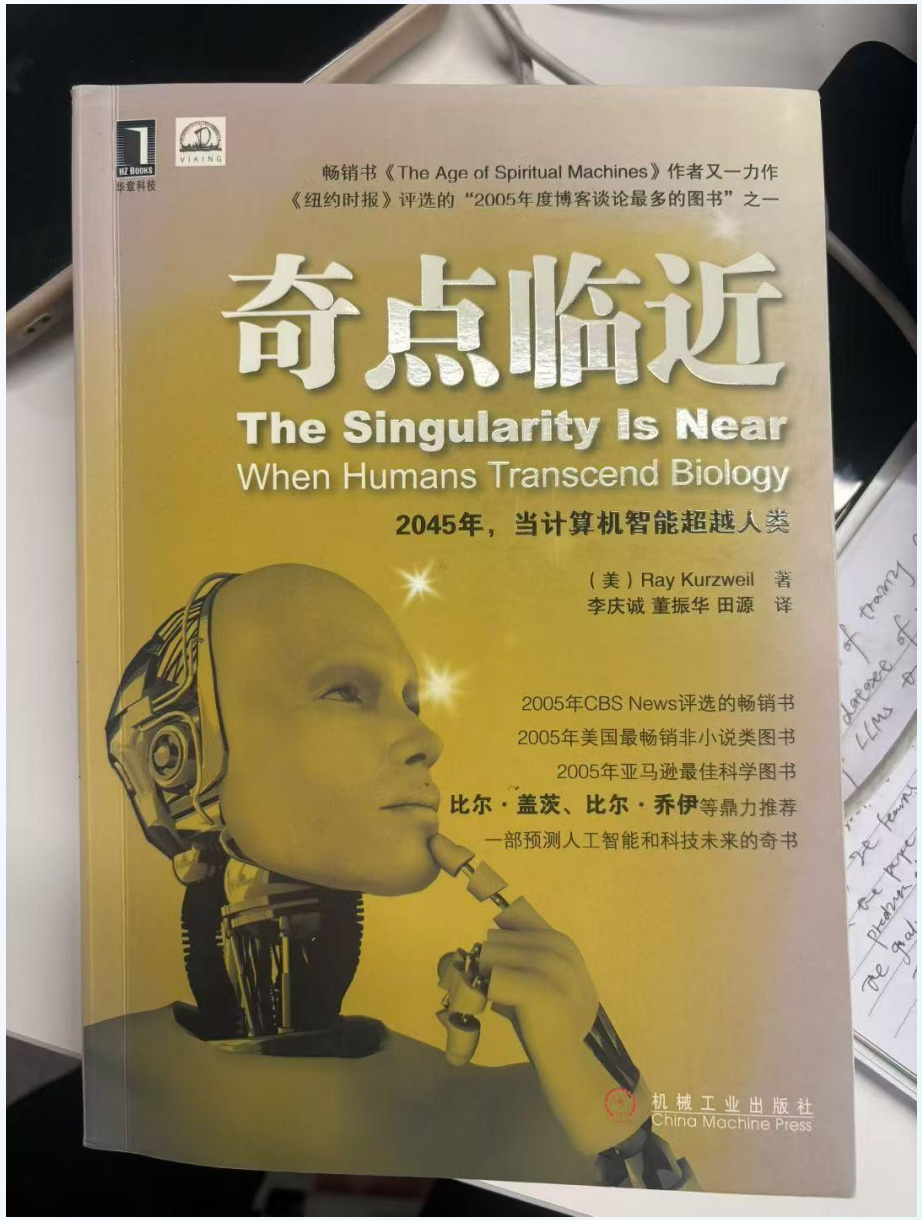
12. Multiple Samples
"Generate 3 different solutions and recommend the best."

13. Structured Output
"Format: ## Analysis\n## Code\n## Tests\n## Complexity”

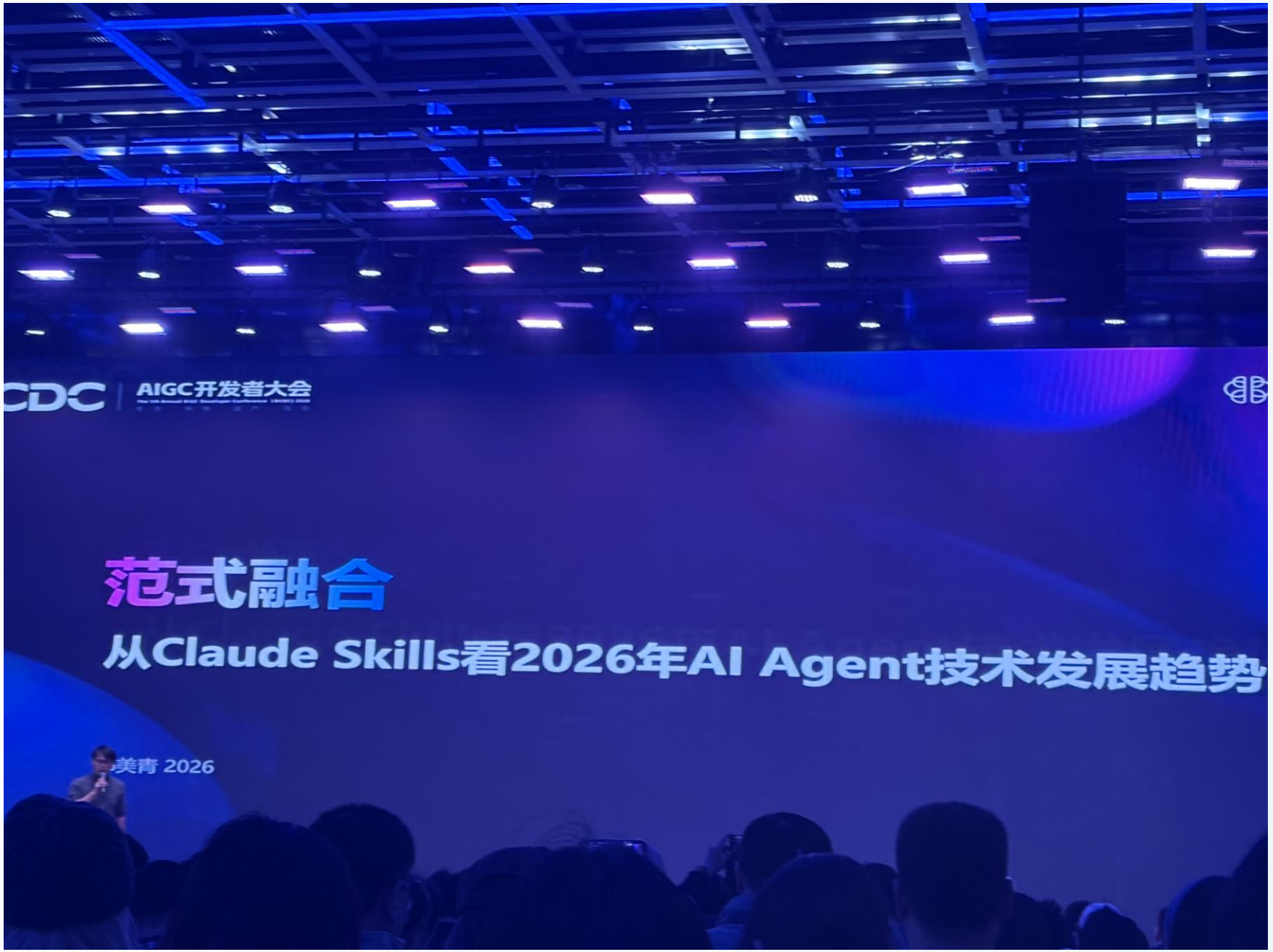
14. We're colleagues working together. I need your honest technical judgment.

Conclusion:

1. 我们已经在奇点上了
2. 我们自身**也是奇点**



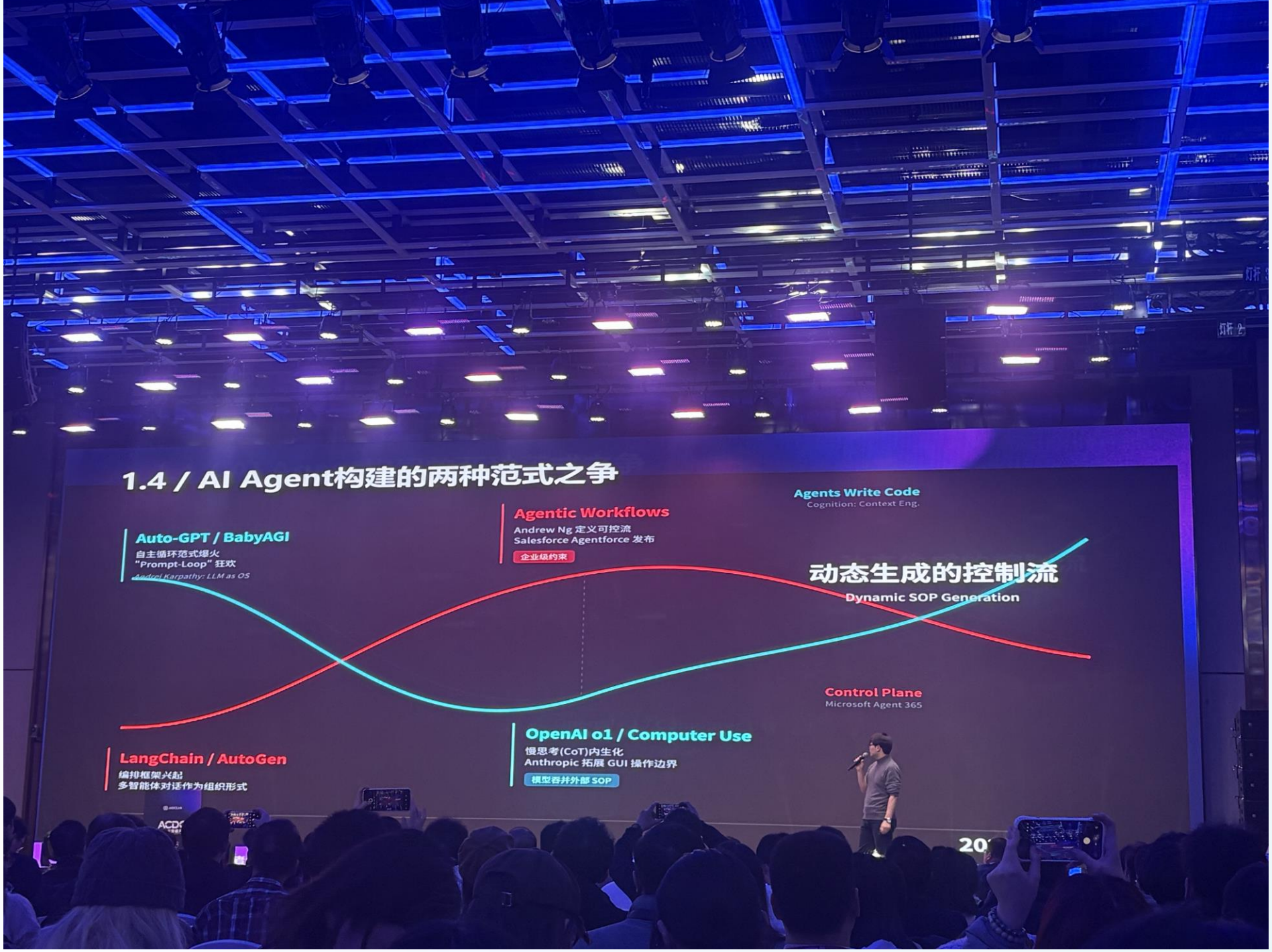
Thanks You Very Much

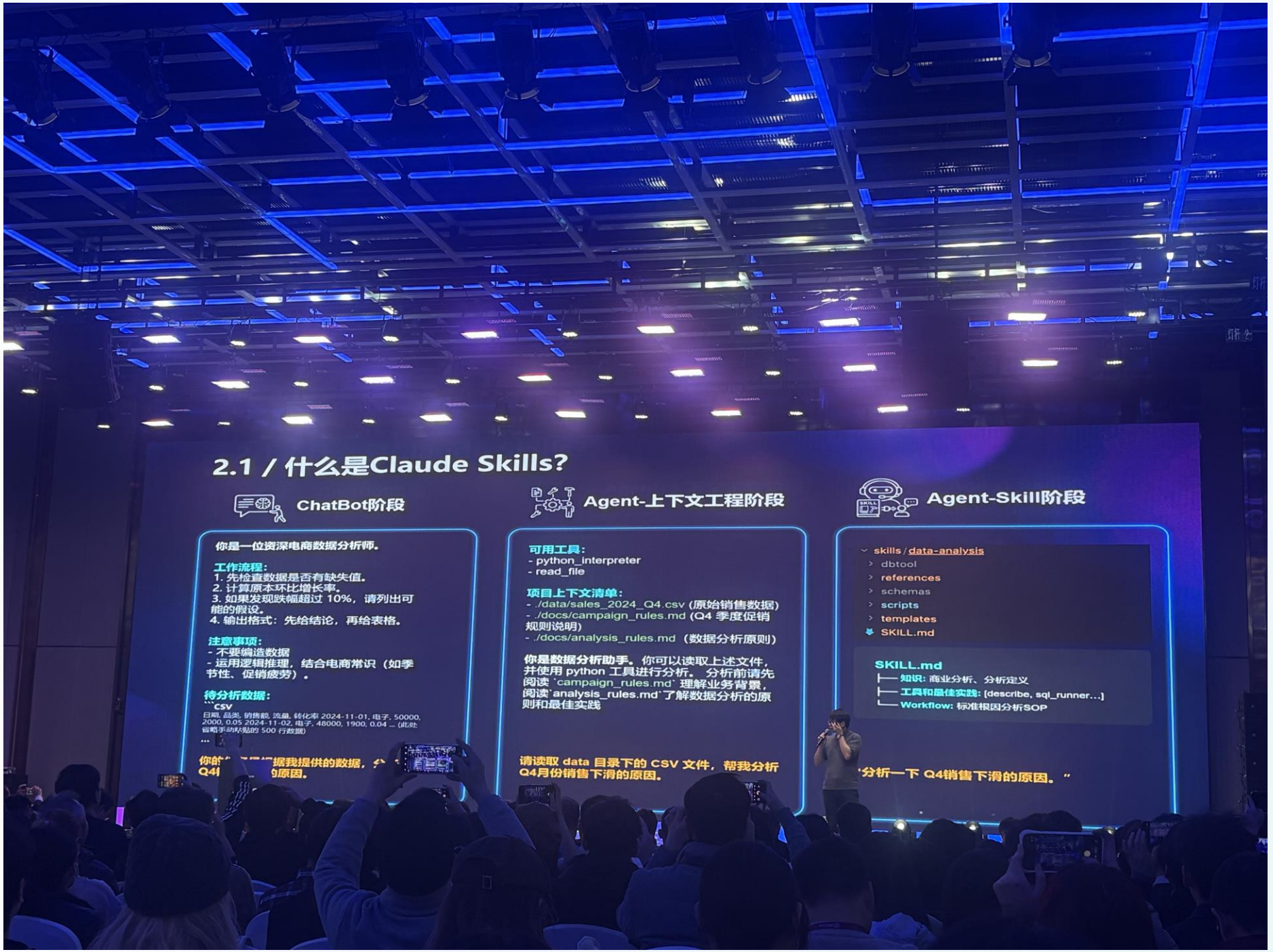

CDC | AIGC开发者大会
范式融合
从Claude Skills看2026年AI Agent技术发展趋势
美青 2026

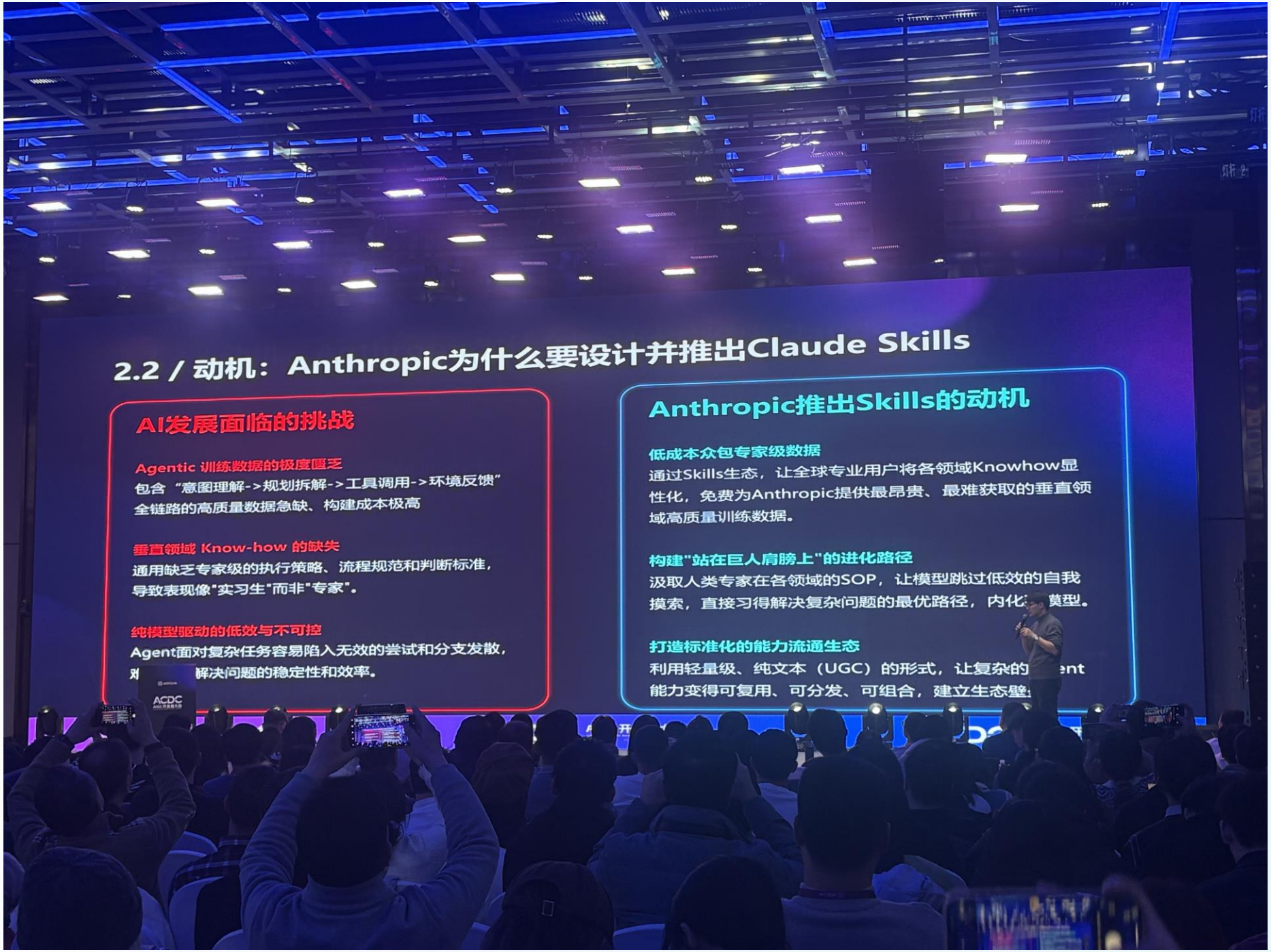

1.1 / AI Agent构建的两种范式
AI
Workflow Agent

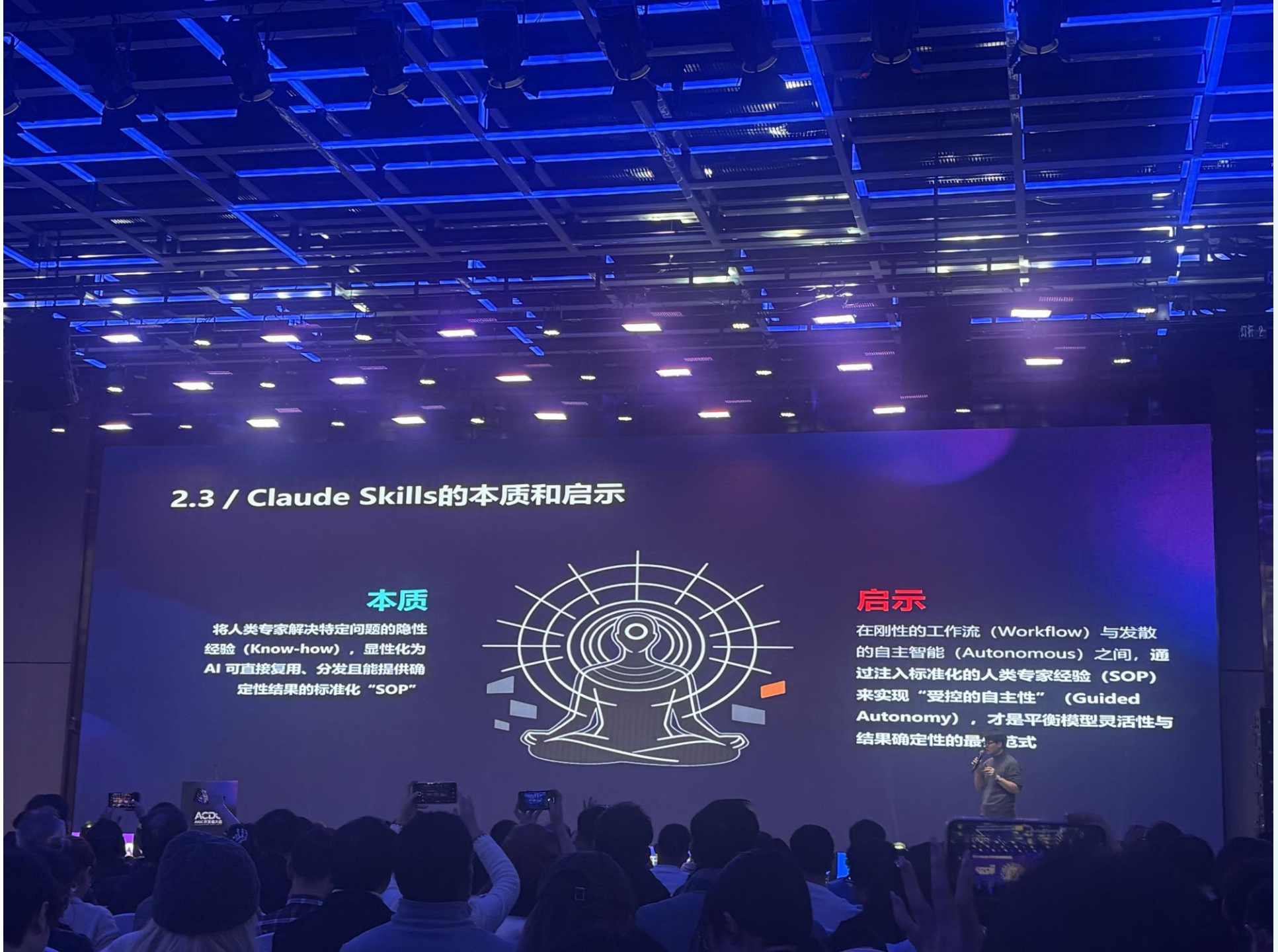

1.2 / Autonomous Agent（自主智能体）
动作 Action
人类
Human
大模型调用
LLM Call
环境
Environment
反馈 Feedback
停止
Stop
模型驱动 (Model-Driven)
放弃硬编码的 if-else 流程。执行路径由 LLM 根据当前状态动态决策，能够应对未预见的场景。
交互闭环 (Interactive Loop)
遵循 感知 - 决策 - 行动 - 观察 的循环模式。不仅仅是生成文本，而是通过工具与真实世界交互。
自我修正 (Self-Correction)
具备从
吴中恢复的能力。当工具调用失败或结果偏差
时
据报错信息自主调整参数重试。


1.3 / WorkFlow Agent（工作流智能体）
Start
Router
LLM Classifier
检索 RAG
代码处理
图像生成
Response
流程驱动 (Flow-Driven)
执行路径是预定义的（如 DAG 图）。LLM 仅作为流程中的节点处理特定任务，不掌握全局控制权。
高确定性 (Determinism)
输出结果具有高度可预测性与一致性。非常适合企业级 SOP（标准作业程序）及合规要求高的场景。
人机协作 (Human-in-the-Loop)
架构清晰，易于
关键节点插入人工审批或强制规则，
确保系统运行
全边界内。


1.4 / AI Agent构建的两种范式之争
Auto-GPT / BabyAGI
自主循环范式爆火
"Prompt-Loop" 狂欢
Andrej Karpathy: LLM as OS
Agentic Workflows
Andrew Ng 定义可控流
Salesforce Agentforce 发布
企业级约束
Agents Write Code
Cognition: Context Eng.
动态生成的控制流
Dynamic SOP Generation
Control Plane
Microsoft Agent 365
OpenAI o1 / Computer Use
慢思考(CoT)内生化
Anthropic 拓展 GUI 操作边界
模型吞并外部 SOP
LangChain / AutoGen
编排框架兴起
多智能体对话作为组织形式


2.1 / 什么是Claude Skills?
ChatBot阶段
你是一位资深电商数据分析师。
工作流程：
1. 先检查数据是否有缺失值。
2. 计算原本环比增长率。
3. 如果发现跌幅超过 10%，请列出可能的假设。
4. 输出格式：先给结论，再给表格。
注意事项：
- 不要编造数据
- 运用逻辑推理，结合电商常识（如季节性、促销疲劳）。
待分析数据：
```csv
日期, 品类, 销售额, 流量, 转化率 2024-11-01, 电子, 50000, 2000, 0.05 2024-11-02, 电子, 48000, 1900, 0.04 ... (此处省略手动粘贴的 500 行数据)
...
Q4 的原因。
Agent-上下文工程阶段
可用工具：
- python_interpreter
- read_file
项目上下文清单：
- ./data/sales_2024_Q4.csv (原始销售数据)
- ./docs/campaign_rules.md (Q4 季度促销规则说明)
- ./docs/analysis_rules.md （数据分析原则）
你是数据分析助手。你可以读取上述文件，并使用 python 工具进行分析。 分析前请先阅读 `campaign_rules.md` 理解业务背景，阅读`analysis_rules.md`了解数据分析的原则和最佳实践
请读取 data 目录下的 CSV 文件，帮我分析Q4月份销售下滑的原因。
Agent-Skill阶段
skills / data-analysis
dbtool
references
schemas
scripts
templates
SKILL.md
SKILL.md
知识: 商业分析、分析定义
工具和最佳实践: [describe, sql_runner...]
Workflow: 标准根因分析SOP
分析一下 Q4销售下滑的原因。"
```


2.2 / 动机：Anthropic为什么要设计并推出Claude Skills
AI发展面临的挑战
Agentic 训练数据的极度匮乏
包含“意图理解->规划拆解->工具调用->环境反馈”
全链路的高质量数据急缺、构建成本极高
垂直领域 Know-how 的缺失
通用缺乏专家级的执行策略、流程规范和判断标准，
导致表现像"实习生"而非"专家"。
纯模型驱动的低效与不可控
Agent面对复杂任务容易陷入无效的尝试和分支发散，
解决问题的稳定性和效率。
Anthropic推出Skills的动机
低成本众包专家级数据
通过Skills生态，让全球专业用户将各领域Knowhow显
性化，免费为Anthropic提供最昂贵、最难获取的垂直领
域高质量训练数据。
构建"站在巨人肩膀上"的进化路径
汲取人类专家在各领域的SOP，让模型跳过低效的自我
摸索，直接习得解决复杂问题的最优路径，内化
模型。
打造标准化的能力流通生态
利用轻量级、纯文本（UGC）的形式，让复杂的
能力变得可复用、可分发、可组合，建立生态壁


2.3 / Claude Skills的本质和启示
本质
将人类专家解决特定问题的隐性经验（Know-how），显性化为AI 可直接复用、分发且能提供确定性结果的标准化“SOP”
启示
在刚性的工作流（Workflow）与发散的自主智能（Autonomous）之间，通过注入标准化的人类专家经验（SOP）来实现“受控的自主性”（Guided Autonomy），才是平衡模型灵活性与结果确定性的最

	Individual Pro	Max	Standard Teams
Cursor	20$	60$	40$
Claude Code	20 $ ~ 45 messages every 5-hour cycle	100$ ~225 messages every 5-hour cycle	200 $ ~800 prompts every 5-hour cycle (20x Pro)
GLM4.7	15 $	30 $	
Qoder	10 $	30$	
CodeX			